

# Ordinals and Typed Lambda Calculus

## Numerical Systems in the Lambda Calculus

Andrés Sicard-Ramírez

Universidad EAFIT

Semester 2018-2

# Church Encoding

---

## Booleans

We encoding the Booleans.

$\text{true} := \lambda x y. x$ , where  $\text{true } M N =_{\beta} M$ .

$\text{false} := \lambda x y. y$ , where  $\text{false } M N =_{\beta} N$ .

# Church Encoding

---

## Definition

For  $n \in \mathbb{N}$  and  $F, M \in \Delta$ , we define

$$\begin{aligned} F^0 M &:= M, \\ F^{n+1} M &:= F (F^n M). \end{aligned}$$

# Church Encoding

---

## Definition

The **Church numerals**, denoted by  $c_n$  with  $n \in \mathbb{N}$ , encode the natural numbers.

- One definition

$$c_n := \lambda f x. f^n x,$$

- Other definition

$$c_0 := \lambda f x. x,$$

$$\text{succ} := \lambda n f x. f (n f x),$$

$$c_{n+1} := \text{succ } c_n.$$

# Church Encoding

---

## Example

$\beta$ -nf's for some numerals.

$$c_0 =_{\beta} \lambda f x. x,$$

$$c_1 =_{\beta} \lambda f x. f x,$$

$$c_2 =_{\beta} \lambda f x. f (f x),$$

$$c_3 =_{\beta} \lambda f x. f (f (f x)).$$

Whiteboard.

# Church Encoding

---

## Addition, multiplication and exponentiation

Encoding for the arithmetic operations.

$\text{add} := \lambda m n f x. m f (n f x)$ , where

$\text{add } c_m c_n =_{\beta} c_{m+n}$ .

$\text{mult} := \lambda m n f x. m (n f) x$   
 $\equiv \lambda m n f. m (n f)$ , where

$\text{mult } c_m c_n =_{\beta} c_{m \times n}$ .

$\text{exp} := \lambda m n f x. (n m) f x$ , where

$\text{exp } c_m c_n =_{\beta} c_{m^n}$ .

# Church Encoding

---

## Testing for zero

A combinator for testing for zero is defined by

$$\text{isZero} := \lambda n. n (\lambda x. \text{false}) \text{true},$$

where

$$\text{isZero } c_0 =_{\beta} \text{true},$$

$$\text{isZero } c_{n+1} =_{\beta} \text{false}.$$

# Church Encoding

---

## Predecessor

A predecessor combinator is defined by

$$\text{pred} := \lambda n f x. n (\lambda g h. h (g f)) (\lambda u. x) (\lambda u. u),$$

where

$$\text{pred } c_0 =_{\beta} c_0,$$

$$\text{pred } c_{n+1} =_{\beta} c_n.$$



# Church Encoding

---

## Recursion

The factorial function is an example of recursive functions on natural numbers following the schemata

$$\begin{aligned}f & : \mathbb{N} \rightarrow A \\f\ 0 & = a \\f\ (S\ n) & = g\ n\ (f\ n)\end{aligned}$$

where  $A$  is a set,  $a \in A$ ,  $g : \mathbb{N} \rightarrow A \rightarrow A$  and  $S$  is the successor function.

## Remark

A reader familiarised with recursion theory will identify the primitive recursion schemata.

# Church Encoding

---

## A recursor for natural numbers

Let  $A$  be a set. From the previous schemata for (primitive) recursive functions, we define a recursor.<sup>†</sup>

$$\begin{aligned}\text{rec} & : (\mathbb{N} \rightarrow A \rightarrow A) \rightarrow A \rightarrow \mathbb{N} \rightarrow A \\ \text{rec } f \ a \ 0 & = a \\ \text{rec } f \ a \ (\text{S } n) & = f \ n \ (\text{rec } f \ a \ n)\end{aligned}$$

---

<sup>†</sup>A higher-order function which allow define recursive functions.

# Church Encoding

---

## Recursor

Since `rec` is also a recursive function, we can define a recursor for numerals using `fix`.

$$\begin{aligned} h &:= \lambda r f a n. (\text{isZero } n) a (f (\text{pred } n) (r f a (\text{pred } n))), \\ \text{rec} &:= \text{fix } h, \end{aligned}$$

where

$$\begin{aligned} \text{rec } f a c_0 &=_{\beta} a, \\ \text{rec } f a c_{n+1} &=_{\beta} f c_n (\text{rec } f a c_n). \end{aligned}$$

# Church Encoding

---

## Recursor

Since `rec` is also a recursive function, we can define a recursor for numerals using `fix`.

$$\begin{aligned}h &:= \lambda r f a n. (\text{isZero } n) a (f (\text{pred } n) (r f a (\text{pred } n))), \\ \text{rec} &:= \text{fix } h,\end{aligned}$$

where

$$\begin{aligned}\text{rec } f a c_0 &=_{\beta} a, \\ \text{rec } f a c_{n+1} &=_{\beta} f c_n (\text{rec } f a c_n).\end{aligned}$$

## Question

What is it necessary for defining `rec`? A fixed-point combinator, (implicit) ‘Boolean’ case analysis, a test for zero and a predecessor function.

# Church Encoding

---

## Example

A combinator for the factorial function

$$\text{fac} : \mathbb{N} \rightarrow \mathbb{N}$$

$$\text{fac } 0 = 1$$

$$\text{fac } (S\ n) = S\ n \times \text{fac } n$$

is defined by

$$\text{fac} := \text{rec } (\lambda x\ y. \text{mult } (\text{succ } x)\ y)\ c_1.$$

# Church Encoding

---

## Definition

A **number-theoretic function** is a function whose signature is

$$\mathbb{N}^k \rightarrow \mathbb{N}, \text{ with } k \in \mathbb{N}.$$

# Church Encoding

---

## Definition

Let  $\varphi$  be a **partial** number-theoretic function  $\varphi : \mathbb{N}^k \rightarrow \mathbb{N}$ . The function  $\varphi$  is  **$\lambda$ -definable** **respect to the Church encoding** iff there exists a  $\lambda$ -term  $F$  such that for all  $n_1, \dots, n_k \in \mathbb{N}$ ,

$$\begin{aligned}\varphi(n_1, \dots, n_k) = a &\Rightarrow F\ c_{n_1} \dots c_{n_k} =_{\beta} c_a, \\ \varphi(n_1, \dots, n_k) \text{ does not exist} &\Rightarrow F\ c_{n_1} \dots c_{n_k} \text{ has no } \beta\text{-nf.}\end{aligned}$$

---

<sup>†</sup>See, e.g. (Barendregt [1981] 2004, Corollary 6.4.6) and (Hindley and Seldin 2008, Theorem 4.23).

# Church Encoding

---

## Definition

Let  $\varphi$  be a **partial** number-theoretic function  $\varphi : \mathbb{N}^k \rightarrow \mathbb{N}$ . The function  $\varphi$  is  **$\lambda$ -definable respect to the Church encoding** iff there exists a  $\lambda$ -term  $F$  such that for all  $n_1, \dots, n_k \in \mathbb{N}$ ,

$$\begin{aligned}\varphi(n_1, \dots, n_k) = a &\Rightarrow F\ c_{n_1} \dots c_{n_k} =_{\beta} c_a, \\ \varphi(n_1, \dots, n_k) \text{ does not exist} &\Rightarrow F\ c_{n_1} \dots c_{n_k} \text{ has no } \beta\text{-nf.}\end{aligned}$$

## Theorem

The Turing-computable functions are  $\lambda$ -definable respect to the Church encoding.<sup>†</sup>

---

<sup>†</sup>See, e.g. (Barendregt [1981] 2004, Corollary 6.4.6) and (Hindley and Seldin 2008, Theorem 4.23).



# Numeral Systems

---

## Definition

A **numeral system** is a sequence of combinators

$$d = d_0, d_1, \dots$$

and combinators  $\text{true}_d$ ,  $\text{false}_d$ ,  $\text{succ}_d$  and  $\text{isZero}_d$  such that for all  $n \in \mathbb{N}$  (Barendregt [1981] 2004, Definition 6.4.1),

$$\begin{aligned}\text{succ}_d d_n &=_{\beta} d_{n+1}, \\ \text{isZero}_d d_0 &=_{\beta} \text{true}_d, \\ \text{isZero}_d d_{n+1} &=_{\beta} \text{false}_d.\end{aligned}$$

# N numeral Systems

---

## Definition

A **numeral system** is a sequence of combinators

$$d = d_0, d_1, \dots$$

and combinators  $\text{true}_d$ ,  $\text{false}_d$ ,  $\text{succ}_d$  and  $\text{isZero}_d$  such that for all  $n \in \mathbb{N}$  (Barendregt [1981] 2004, Definition 6.4.1),

$$\begin{aligned}\text{succ}_d d_n &=_{\beta} d_{n+1}, \\ \text{isZero}_d d_0 &=_{\beta} \text{true}_d, \\ \text{isZero}_d d_{n+1} &=_{\beta} \text{false}_d.\end{aligned}$$

## Example

The Church numerals  $c = c_0, c_1, \dots$  and the combinators  $\text{true}$ ,  $\text{false}$ ,  $\text{succ}$  and  $\text{isZero}$  are a numeral system.

# Numeral Systems

---

## Notation

We shall denote by  $\mathbf{d}$  a numeral system  $d_0, d_1, \dots, \text{true}_{\mathbf{d}}, \text{false}_{\mathbf{d}}, \text{succ}_{\mathbf{d}}$  and  $\text{isZero}_{\mathbf{d}}$ .

# N numeral Systems

---

## Notation

We shall denote by  $\mathbf{d}$  a numeral system  $\mathbf{d}_0, \mathbf{d}_1, \dots, \text{true}_{\mathbf{d}}, \text{false}_{\mathbf{d}}, \text{succ}_{\mathbf{d}}$  and  $\text{isZero}_{\mathbf{d}}$ .

## Lambda definable functions on a numeral system

Let  $\mathbf{d}$  be a numeral system and let  $\varphi$  be a partial number-theoretic function  $\varphi : \mathbb{N}^k \rightarrow \mathbb{N}$ . The function  $\varphi$  is  **$\lambda$ -definable respect to the numeral system  $\mathbf{d}$**  iff there exists a  $\lambda$ -term  $F$  such that for all  $n_1, \dots, n_k \in \mathbb{N}$ ,

$$\begin{aligned}\varphi(n_1, \dots, n_k) = a &\Rightarrow F \mathbf{d}_{n_1} \dots \mathbf{d}_{n_k} =_{\beta} \mathbf{d}_a, \\ \varphi(n_1, \dots, n_k) \text{ does not exist} &\Rightarrow F \mathbf{d}_{n_1} \dots \mathbf{d}_{n_k} \text{ has no } \beta\text{-nf.}\end{aligned}$$

# Numeral Systems

---

## Definition

A numeral system  $d$  is **adequate** iff all the Turing-computable functions are  $\lambda$ -definable with respect to  $d$  (Barendregt [1981] 2004, Definition 6.4.2ii).

# Numeral Systems

---

## Definition

A numeral system  $d$  is **adequate** iff all the Turing-computable functions are  $\lambda$ -definable with respect to  $d$  (Barendregt [1981] 2004, Definition 6.4.2ii).

## Example

The Church numeral system is adequate.

# Numeral Systems

---

## Theorem

A numeral system  $d$  is adequate iff there exists a combinator  $\text{pred}_d$  such that for all  $n \in \mathbb{N}$  (Barendregt [1981] 2004, Theorem 6.4.3),

$$\text{pred}_d d_{n+1} =_{\beta} d_n.$$

# Numeral Systems

---

## Theorem

A numeral system  $\mathbf{d}$  is adequate iff there exists a combinator  $\text{pred}_{\mathbf{d}}$  such that for all  $n \in \mathbb{N}$  (Barendregt [1981] 2004, Theorem 6.4.3),

$$\text{pred}_{\mathbf{d}} \mathbf{d}_{n+1} =_{\beta} \mathbf{d}_n.$$

## Remark

There are various adequate numeral systems in the literature. See, e.g. Barendregt ([1981] 2004), Goldberg (2000) and Jansen (2013).



## References

---

- H. P. Barendregt [1981] (2004). The Lambda Calculus. Its Syntax and Semantics. Revised edition, 6th impression. Vol. 103. Studies in Logic and the Foundations of Mathematics. Elsevier (cit. on pp. 15–18, 21–24).
- Mayer Goldberg (2000). An Adequate and Efficient Left-Associated Binary Numeral System in the  $\Lambda$ -Calculus. Journal of Functional Programming 10.6, pp. 607–623. DOI: [10.1017/S0956796800003804](https://doi.org/10.1017/S0956796800003804) (cit. on pp. 23, 24).
- J. Roger Hindley and Jonathan P. Seldin (2008). Lambda-Calculus and Combinators. An Introduction. Cambridge University Press (cit. on pp. 15, 16).
- Jan Martin Jansen (2013). Programming in the  $\Lambda$ -Calculus: From Church to Scott and Back. In: The Beauty of Functional Code. Ed. by Peter Achten and Pieter Koopman. Vol. 8106. Lecture Notes in Computer Science. Springer, pp. 168–180. DOI: [10.1007/978-3-642-40355-2\\_12](https://doi.org/10.1007/978-3-642-40355-2_12) (cit. on pp. 23, 24).